介绍

SSRF，全称 **Server-Side Request Forgery**，即**服务器端请求伪造**。顾名思义，这是一种由攻击者控制，但由**服务器自身**发起的请求。攻击者利用这一漏洞，能够强制服务器去访问其**原本不应该访问的资源**。

SSRF 的核心原理

SSRF 漏洞的产生，通常是因为应用程序从用户提供的 URL 地址中获取数据，但**没有对用户输入的 URL 进行充分的验证和过滤**。当服务器去访问这个用户提供的 URL 时，攻击者就可以利用这一点，构造恶意的 URL，让服务器去请求一些攻击者想访问但直接访问不到的资源。

简单来说，SSRF 的核心原理就是：**让受信任的服务器变成攻击者的“代理”**。

它是如何发生的?

想象一下一个功能，比如“通过 URL 加载图片”或者“从远程服务器获取数据”。用户在前端输入一个 URL，服务器会去访问这个 URL，然后把内容返回给用户。

如果这个 URL 参数没有被严格验证，攻击者就可以这样操作：

1. **用户提供恶意 URL**: 攻击者构造一个指向内部系统（例如，`http://127.0.0.1/admin` 或 `http://internal-db/config`）或外部恶意服务器的 URL。
2. **服务器发起请求**: 应用程序后端接收到这个恶意 URL，并且不加区分地信任它，然后使用服务器自身的权限和网络环境去发起请求。
3. **获取敏感信息或执行操作**:
    - **访问内部网络资源**: 服务器可能会访问到只有它才能访问的内网 IP 地址、端口或服务，比如内部管理接口、数据库、云服务元数据等。
    - **攻击其他外部服务**: 服务器也可能被用来扫描或攻击其他外部的、受防火墙保护的服务。
    - **利用内部 API**: 通过服务器的请求，调用内部未对外开放的 API 接口，从而执行一些敏感操作。
    - **探测端口**: 探测服务器所在网络的开放端口情况。

可能照成SSRF的api

1. HttpClient (Apache)

介绍：Apache HttpClient是一个功能强大的HTTP客户端库，支持HTTP协议的最新标准。

功能: Apache HttpClient (同步) 和 HttpAsyncClient (异步) 是 Apache 软件基金会开发的功能强大、高度可定制的 HTTP 客户端库。它们提供了比 Java 内置 `HttpURLConnection` 更丰富的特性，如连接池、认证、代理、重试机制等。

支持的协议: 主要专注于 HTTP/HTTPS 协议：

- `http`：支持。
- `https`：支持。

SSRF 风险: 高。由于其主要用于发起 HTTP/HTTPS 请求，如果用户提供的 URL 没有经过严格验证，攻击者可以利用它发起对内网或元数据服务的 HTTP/HTTPS 请求。通常情况下，它不会支持 `file://` 或 `gopher://` 等非 HTTP 协议，但这并不意味着没有风险。

考拉安全内部培训资料

```
// 有SSRF漏洞的Spring Controller示例
@RestController
public class HttpClientController {
    @GetMapping("/httpclient")
    public String httpClientSSRF(@RequestParam String url) throws Exception {
        CloseableHttpClient client = HttpClients.createDefault();
        HttpGet request = new HttpGet(url); // 用户可控的URL

        try (CloseableHttpResponse response = client.execute(request)) {
            return EntityUtils.toString(response.getEntity());
        }
    }
}
```

2. HttpAsyncClient (Apache)

介绍：Apache HttpAsyncClient是HttpClient的异步版本，支持非阻塞IO。

```
@RestController
public class AsyncHttpClientController {
    @GetMapping("/async")
    public CompletableFuture<String> asyncHttpClientSSRF(@RequestParam String
url) {
        CloseableHttpAsyncClient client = HttpAsyncClients.createDefault();
        client.start();

        HttpGet request = new HttpGet(url); // 用户可控的URL
        CompletableFuture<String> future = new CompletableFuture<>();

        client.execute(request, new FutureCallback<HttpResponse>() {
            public void completed(HttpResponse response) {
                future.complete(EntityUtils.toString(response.getEntity()));
            }
            // 其他回调方法省略...
        });

        return future;
    }
}
```

菜狗安全内部培训资料

3. java.net.URLConnection/HttpURLConnection

介绍：Java标准库中的HTTP客户端实现。

功能: `URLConnection` 是一个抽象类，代表应用程序和 URL 之间的通信链接。`HttpURLConnection` 是其子类，专门用于处理 HTTP/HTTPS 请求。它们是 Java 标准库中进行网络请求的基础。

支持的协议:

- `HttpURLConnection`: 仅支持 `http` 和 `https` 协议。
- `URLConnection`: 作为更通用的父类，它继承了 `URL` 类所能解析的协议能力，理论上可以支持所有 JRE 默认支持的协议，如 `http`, `https`, `file`, `ftp`, `jar`, `gopher`, `dict` 等。实际可用性取决于 JVM 内部注册的 `URLStreamHandler`。

SSRF 风险: 极高。可以直接发起对任意协议和地址的请求，是 SSRF 漏洞最常见的利用点。

```
@RestController
public class URLConnectionController {
    @GetMapping("/urlconnection")
    public String urlConnectionSSRF(@RequestParam String url) throws Exception {
        URL obj = new URL(url); // 用户可控的URL
        HttpURLConnection con = (HttpURLConnection) obj.openConnection();

        BufferedReader in = new BufferedReader(new
InputStreamReader(con.getInputStream()));
        String inputLine;
        StringBuilder response = new StringBuilder();

        while ((inputLine = in.readLine()) != null) {
            response.append(inputLine);
        }
        in.close();

        return response.toString();
    }
}
```

4. java.net.URL

介绍：Java标准库中用于表示统一资源定位器的类

功能: `URL` 类是 Java 标准库中用于表示统一资源定位符（Uniform Resource Locator）的核心类。它本身**不直接发起网络请求**，但它是创建 `URLConnection` 或 `HttpURLConnection` 的基础。

支持的协议: `URL` 类能够识别和解析多种协议，其是否能实际连接取决于底层 `URLStreamHandler` 的实现。Java 运行时环境（JRE）默认通常支持以下协议：

- `http`: 超文本传输协议。
- `https`: 安全超文本传输协议。
- `file`: 文件协议，用于访问本地文件系统资源。这是 SSRF 中常被利用来读取本地文件的协议。
- `ftp`: 文件传输协议。
- `jar`: JAR 文件协议，用于访问 JAR 文件内部的资源。
- `mailto`: 邮件协议（通常用于打开邮件客户端，不涉及网络连接）。
- `gopher`: Gopher 协议，一种早期的分布式文档传输协议，现在很少使用，但在 SSRF 中常被利用进行内网探测或发送自定义数据包。
- `dict`: 字典协议，用于访问字典服务，在 SSRF 中也常被利用进行端口扫描或信息探测。

SSRF 风险: 极高。作为许多网络请求的基础，如果用户能控制 `URL` 对象的创建，就可能导致对任意协议和地址的访问。

长亭内部培训资料

```
@RestController
public class URLController {
    @GetMapping("/url")
    public String urlSSRF(@RequestParam String url) throws Exception {
        URL obj = new URL(url); // 用户可控的URL
        try (InputStream is = obj.openStream()) {
            return new String(is.readAllBytes());
        }
    }
}
```

5. java.net.Socket

介绍：Java标准库中的原始套接字实现，可用于任意协议通信。

功能: `Socket` 类提供了实现客户端套接字（或称为“套接字”）的接口。它是网络通信的**最底层 API** 之一，允许程序直接通过 IP 地址和端口号建立 TCP 连接。

支持的协议: `Socket` 本身**不处理高层协议**（如 HTTP、FTP）。它只负责建立和维护 TCP 连接。一旦连接建立，应用程序可以通过 Socket 的输入/输出流发送和接收**任何基于 TCP 的原始字节流**。这意味着它可以用于实现任何基于 TCP 的协议，例如：

- **HTTP/HTTPS** (但需要手动实现协议解析和构建)
- **FTP**
- **SMTP/POP3/IMAP**
- **SSH**
- **Telnet**
- **任意自定义 TCP 协议**

SSRF 风险: 极高。由于能够连接到任意 IP 和端口，并发送自定义数据，攻击者可以利用它进行精确的端口扫描、指纹识别甚至直接发送恶意负载到内部服务。

```
@RestController
public class SocketController {
    @GetMapping("/socket")
    public String socketSSRF(@RequestParam String host, @RequestParam int port)
throws Exception {
        try (Socket socket = new Socket(host, port); // 用户可控的主机和端口
             PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
             BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()))) {

            out.println("GET / HTTP/1.1\r\nHost: " + host + "\r\n\r\n");
            return in.lines().collect(Collectors.joining("\n"));
        }
    }
}
```

6. OkHttp

介绍：Square公司开发的高效HTTP客户端。

柴狗安全内部培训资料

功能: OkHttp 是 Square 公司开发的一个现代、高效的 HTTP 和 HTTP/2 客户端。它以简洁的 API、对 HTTP/2 和 SPDY 的原生支持、连接池、缓存等特性而闻名。

支持的协议: 主要专注于 HTTP/HTTPS 协议，并支持高级特性：

- `http`: 支持。
- `https`: 支持。
- **HTTP/2**: 支持。
- **WebSocket**: 支持。

SSRF 风险: 高。与 Apache HttpClient 类似，其主要风险在于未经严格验证的 HTTP/HTTPS 请求可能导致对内网资源的访问。同样，通常不支持 `file://` 等非 HTTP 协议。

```
@RestController
public class OkHttpController {
    @GetMapping("/okhttp")
    public String okHttpSSRF(@RequestParam String url) throws Exception {
        OkHttpClient client = new OkHttpClient();
        Request request = new Request.Builder()
            .url(url) // 用户可控的URL
            .build();

        try (Response response = client.newCall(request).execute()) {
            return response.body().string();
        }
    }
}
```

7. ImageIO

介绍：Java标准库中用于图像处理的类，可能被滥用于SSRF。

功能: `javax.imageio.ImageIO` 是 Java 标准库中用于读取和写入图像的 API。例如，`ImageIO.read(URL url)` 可以直接从 URL 读取图片数据。

支持的协议: 当 `ImageIO.read(URL url)` 方法接收 `URL` 对象时，其底层会使用 `URL.openConnection()` 来获取输入流，因此它继承了 `java.net.URL` 和 `java.net.URLConnection` 所支持的所有协议。

- `http`, `https`: 用于从 Web 读取图片。
- `file`: 用于从本地文件系统读取图片。
- `ftp`, `gopher`, `dict`: 理论上也支持，但实际情况可能因具体 JVM 实现和图片格式而异，但其底层发起连接的能力是存在的。

SSRF 风险: 极高。这是典型的 SSRF 场景，攻击者可以利用 `file://` 协议读取服务器上的任意文件，或者利用 `http://`、`https://` 协议探测内网。

蚂蚁安全内部培训资料

```
@RestController
public class ImageIOController {
    @GetMapping("/imageio")
    public void imageIoSSRF(@RequestParam String url) throws Exception {
        BufferedImage image = ImageIO.read(new URL(url)); // 用户可控的URL
        // 处理图像...
    }
}
```

8. Hutool

介绍：Hutool是一个Java工具包，包含HttpUtil等工具类。

功能: Hutool 是一个 Java 工具库，它封装了许多常用的功能，包括网络请求。Hutool 的网络模块（`cn.hutool.http`）通常基于 `HttpURLConnection` 或 Apache HttpClient 等底层库。

支持的协议: Hutool 的 HTTP 客户端模块主要支持：

- `http`: 支持。
- `https`: 支持。
- **其他协议**: Hutool 本身是工具库，其对其他协议的支持取决于它底层所使用的 Java API 或第三方库。如果其封装了 `URL` 或 `URLConnection` 的直接使用，那么它也可能间接支持 `file`、`ftp` 等协议。**通常情况下，其 `HttpRequest` 和 `HttpResponse` 类主要聚焦于 HTTP/HTTPS。**

SSRF 风险: 高。如果直接将用户提供的 URL 传递给 Hutool 的网络请求方法，就可能导致 SSRF。需要查看其具体封装方式和配置，以确定是否会处理非 HTTP 协议。

```
@RestController
public class HutoolController {
    @GetMapping("/hutool")
    public String hutoolSSRF(@RequestParam String url) {
        return HttpUtil.get(url); // 用户可控的URL
    }
}
```

9. Jsoup

介绍：HTML解析库，常用于网络爬虫。

功能: Jsoup 是一个用于解析 HTML 的 Java 库。它提供了非常方便的 API 来从 URL、文件或字符串中解析 HTML，提取数据，以及修改 HTML。其从 URL 读取 HTML 的功能涉及网络请求。

支持的协议: 当 Jsoup 使用 `Jsoup.connect(String url)` 方法时，它主要支持：

- `http`: 支持。
- `https`: 支持。
- `file`: 当 URL 为 `file:///` 格式时，Jsoup 也能识别并从本地文件加载 HTML。

SSRF 风险: 高。由于支持从 URL 读取内容，且能够处理 `file://` 协议，如果直接将用户提供的 URL 传递给 Jsoup，就可能导致文件读取或内网探测的 SSRF。

菜狗安全内部培训资料

```
@RestController
public class JsoupController {
    @GetMapping("/jsoup")
    public String jsoupSSRF(@RequestParam String url) throws Exception {
        return Jsoup.connect(url) // 用户可控的URL
                    .get()
                    .html();
    }
}
```

10. RestTemplate

介绍：Spring框架提供的HTTP客户端。

功能: `RestTemplate` 是 Spring Framework 提供的一个同步 HTTP 客户端，用于简化 RESTful 服务调用。它在 Spring 5.0 以后被 `WebClient` 取代，但仍然广泛使用。其底层可以配置为使用不同的 HTTP 客户端库，如 `HttpURLConnection`、Apache HttpClient、OkHttp 等。

支持的协议: 依赖于其底层配置的 HTTP 客户端工厂 (ClientHttpRequestFactory)。默认情况下通常是：

- `http`: 支持。
- `https`: 支持。
- **其他协议**: 如果底层配置的 `ClientHttpRequestFactory` 使用的是 `HttpURLConnectionFactory` (基于 `java.net.HttpURLConnection`)，那么它将仅支持 `http` 和 `https`。如果配置了支持其他协议的第三方客户端（这种情况较少见），则可能支持更多。

SSRF 风险: 高。由于 `RestTemplate` 用于向外部或内部服务发起 HTTP/HTTPS 请求，如果用户能控制请求的 URL，就会导致 SSRF。它通常不会直接支持 `file://` 这种协议，但对 HTTP/HTTPS 的内网访问能力是其主要风险点。

```
@RestController
public class RestTemplateController {
    @GetMapping("/resttemplate")
    public String restTemplateSSRF(@RequestParam String url) {
        RestTemplate restTemplate = new RestTemplate();
        return restTemplate.getForObject(url, String.class); // 用户可控的URL
    }
}
```

审计思路

和sql注入一样，先判断依赖，然后关键词搜索，在上面提到的10个发送请求的api中

```
java.net.URLConnection
java.net.HttpURLConnection
java.net.URL
java.net.Socket
ImageIO

java11
java.net.http
```

柔韵安全内部培训资料

这几个库是java自带的,已经封装在JDK中了，不需要而外引入

```
HttpClient
HttpAsyncClient
OkHttp
Hutool
Jsoup
RestTemplate
```

而这几个不是自带的，如果要使用他们的话，需要添加maven依赖，拉取到本地

那么我们在挖掘SSRF漏洞的时候就可以优先在pom.xml文件中搜索这几个第三方库，发现有引入对应库，那么就有可能使用它，再去全局搜索这个库发送请求的关键字，定位代码。没有的话就搜索原生的

黑盒测试常见点

1. 文件处理功能

- **文件导入/导出**：支持从URL导入文件的功能
- **图片处理**：头像设置、图片裁剪等支持URL加载图片
- **文档预览**：Office/PDF在线预览功能
- **文件下载**：通过URL下载文件到服务器

2. 数据获取功能

- **网页抓取**：URL预览、网页截图功能
- **RSS订阅**：通过URL获取订阅内容
- **API测试**：允许用户输入API端点进行测试
- **数据采集**：从指定URL采集数据

3. 网络相关功能

- **链接检测**：死链检查、网站可用性检测
- **端口扫描**：服务器状态检测功能
- **Ping/ICMP测试**：网络连通性测试
- **DNS查询**：域名解析功能

4. 云服务相关功能

- **云存储集成**：AWS S3、阿里云OSS等URL处理
- **Webhook配置**：回调URL设置
- **CI/CD集成**：从URL获取构建脚本
- **容器镜像拉取**：通过URL指定镜像源

5. 其他常见功能

- **支付回调**：支付结果回调URL
- **邮件服务**：通过URL加载远程内容

类狗安全内部培训资料

- **OAuth认证**：第三方登录回调处理
- **视频/音频流**：通过URL加载媒体资源

实战案例

关键词

```
def check_ssrf(file_content):
    """检测SSRF相关技术"""
    print("\n==== SSRF技术检测 ====")
    ssrf_techs = {
        "httpclient": ("Apache HttpClient", "execute("),
        "httpasyncclient": ("Apache HttpAsyncClient", "openConnection()"),
        "okhttp": ("OkHttp", ".execute()"),
        "hutool-all": ("Hutool", "HttpUtil.request()"),
        "jsoup": ("Jsoup", "Jsoup.connect("),
        "RestTemplate": ("Spring RestTemplate", "getForObject("),
        "WebClient": ("Spring WebClient", "get().uri(")
    }
```

```
SSRF漏洞检测点:
openConnection()
openStream()
Socket(
ImageIO.read(
execute(
```

菜狗安全内部培训资料